

3. ТИПЫ (TYPES)

Теперь, когда основы позади, мы рассмотрим систему типов Rust. Мы пропустим базовые вещи, рассмотренные в книге «The Rust Programming Language», и вместо этого нырнём с головой в то, как разные типы располагаются в памяти, во все тонкости трейтов (traits) и ограничений трейтов (trait bounds), экзистенциальные типы (existential types) и правила использования типов через границы крейтов (crate boundaries).

Типы в памяти

У каждого значения в Rust есть тип. Типы служат многим целям в Rust, как мы увидим в этой главе, но одна из их самых фундаментальных ролей — сообщать вам, как интерпретировать биты памяти. Например, последовательность битов `0b10111101` (записанная в шестнадцатеричной нотации как `0xBD`) сама по себе ничего не означает, пока вы не присвоите ей тип. При интерпретации под типом `u8` эта последовательность битов является числом 189. При интерпретации под типом `i8` это `-67`. Когда вы определяете свои собственные типы, работа компилятора — определить, где каждая часть определённого типа располагается в представлении этого типа в памяти. Где в последовательности битов появляется каждое поле вашей структуры? Где хранится дискриминант (discriminant) вашего перечисления (enum)? Важно понимать, как работает этот процесс, по мере того как вы начинаете писать более продвинутый код на Rust, поскольку эти детали влияют как на корректность, так и на производительность вашего кода.

Выравнивание (Alignment)

Прежде чем мы поговорим о том, как определяется представление типа в памяти, нам сначала нужно обсудить понятие *выравнивания* (alignment), которое диктует, где могут храниться байты типа. Как только представление типа определено, вы могли бы подумать, что можно взять любую произвольную локацию памяти и интерпретировать хранящиеся там байты как этот тип. Хотя в теоретическом смысле это верно, на практике аппаратное обеспечение также ограничивает то, где может быть размещён данный тип. Наиболее очевидный пример этого — то, что указатели указывают на *байты*, а не на *биты*. Если бы вы разместили значение типа `T`, начиная с бита 4 памяти

вашего компьютера, у вас не было бы способа сослаться на его локацию; вы можете создать указатель, указывающий только на байт 0 или байт 1 (бит 8). По этой причине все значения, независимо от их типа, должны начинаться на границе байта. Мы говорим, что все значения должны быть как минимум *выровнены по байту* (byte-aligned) — они должны быть размещены по адресу, кратному 8 битам.

Некоторые значения имеют более строгие правила выравнивания, чем просто выравнивание по байту. В процессоре и системе памяти доступ к памяти часто осуществляется блоками больше одного байта. Например, на 64-битном процессоре доступ к большинству значений осуществляется кусками по 8 байт (64 бита), при этом каждая операция начинается с адреса, выровненного по 8 байтам. Это называется *размером слова* (word size) процессора. Затем процессор использует некоторую хитрость для обработки чтения и записи меньших значений или значений, охватывающих границы этих кусков.

Где возможно, вы хотите гарантировать, что аппаратное обеспечение может работать в своём «родном» выравнивании. Чтобы понять почему, рассмотрите, что происходит, если вы пытаетесь прочитать `i64`, который начинается в середине 8-байтового блока (то есть указатель на него не выровнен по 8 байтам). Аппаратному обеспечению придётся выполнить два чтения — одно из второй половины первого блока, чтобы получить начало `i64`, и одно из первой половины второго блока, чтобы прочитать остаток `i64`, — а затем склеить результаты вместе. Это не очень эффективно. Поскольку операция распределена по нескольким обращениям к нижележащей памяти, вы также можете получить странные результаты, если память, из которой вы читаете, конкурентно записывается другим потоком. Вы можете прочитать первые 4 байта до того, как произошла запись другого потока, и вторые 4 байта после, что приведёт к испорченному значению.

Операции над данными, которые не выровнены, называются *невывровненными обращениями* (misaligned accesses) и могут привести к плохой производительности и проблемам параллелизма. По этой причине многие операции процессора требуют или настоятельно предпочитают, чтобы их аргументы были *естественно выровнены* (naturally aligned). Естественно выровненное значение — это такое, чьё выравнивание совпадает с его размером. Так, например, для 8-байтовой загрузки предоставленный адрес должен быть выровнен по 8 байтам.

Поскольку выровненные обращения в целом быстрее и обеспечивают более сильную семантику согласованности, компилятор пытается использовать их везде, где это возможно. Он делает это, давая каждому типу выравнивание, вычисленное на основе типов, которые он содержит. Встроенные значения обычно выровнены по своему размеру, так что `u8` выровнен по байту, `u16` выровнен по 2 байтам, `u32` выровнен по 4 байтам, а `u64` выровнен по 8 байтам. Сложные типы — типы, которые содержат другие типы, — обычно получают наибольшее выравнивание любого из типов, которые они содержат. Например, тип, содержащий `u8`, `u16` и `u32`, будет выровнен по 4 байтам из-за `u32`.

Раскладка (Layout)

Теперь, когда вы знаете о выравнивании, мы можем исследовать, как компилятор решает, каким будет представление типа в памяти, известное как *раскладка* (layout) типа. По умолчанию компилятор Rust даёт очень мало гарантий о том, как он раскладывает типы, что является плохой отправной точкой для понимания нижележащих принципов. К счастью, Rust предоставляет атрибут `repr`, который вы можете добавить к определениям своих типов, чтобы запросить конкретное представление в памяти для этого типа. Наиболее распространённый, который вы увидите (если вообще увидите хоть какой-то), — это `repr(C)`. Как следует из названия, он раскладывает тип способом, совместимым с тем, как компилятор C или C++ разложил бы тот же тип. Это полезно при написании кода на Rust, который взаимодействует с другими языками с помощью интерфейса внешних функций (foreign-function interface), о котором мы поговорим в главе 12, поскольку Rust сгенерирует раскладку, соответствующую ожиданиям компилятора другого языка. Поскольку раскладка C предсказуема и не подвержена изменениям, `repr(C)` также полезна в небезопасных (unsafe) контекстах, если вы работаете с сырыми указателями в тип или если вам нужно приводить (cast) между двумя разными типами, о которых вы знаете, что они имеют одни и те же поля. И, конечно, она идеально подходит, чтобы сделать наши первые шаги в алгоритмы раскладки.

ПРИМЕЧАНИЕ Ещё одно полезное представление — это `repr(transparent)`, которое можно использовать только с типами с единственным полем, и оно гарантирует, что раскладка внешнего типа точно такая же, как у внутреннего типа. Это пригодится в сочетании с паттерном «newtype», где вы можете захотеть оперировать представлениями в памяти некоторых `struct A` и `struct NewA(A)`, как если бы они были одинаковыми. Без `repr(transparent)` компилятор Rust не гарантирует, что они будут иметь одинаковую раскладку.

Итак, давайте посмотрим, как компилятор разложил бы конкретный тип с `repr(C)`: тип `Foo` в листинге 3-1. Как вы думаете, как компилятор разложил бы это в памяти?

```
#[repr(C)]
struct Foo {
    tiny: bool,
    normal: u32,
    small: u8,
    long: u64,
    short: u16,
}
```

Листинг 3-1: Выравнивание влияет на раскладку.

Сначала компилятор видит поле `tiny`, чей логический размер составляет 1 бит (`true` или `false`). Но поскольку процессор и память оперируют в терминах байтов, `tiny` получает 1 байт в представлении в памяти. Далее, `normal` — это 4-байтовый тип, так что мы хотим, чтобы он был выровнен по 4 байтам. Но даже если `Foo` выровнен, 1 байт, который мы выделили `tiny`, заставит `normal` промахнуться мимо своего выравнивания. Чтобы это исправить, компилятор вставляет 3 байта *набивки* (padding) — байты с неопределённым значением, которые игнорируются в пользовательском коде, — в представление в памяти между `tiny` и `normal`. В набивку не помещаются никакие значения, но она всё же занимает место.

Для следующего поля, `small`, выравнивание простое: это 1-байтовое значение, а текущее байтовое смещение в структуру составляет $1 + 3 + 4 = 8$. Это уже выровнено по байту, так что `small` может идти сразу после `normal`. Однако с `long` у нас снова проблема. Мы теперь находимся на смещении $1 + 3 + 4 + 1 = 9$ байт внутри `Foo`. Если `Foo` выровнен, то `long` не выровнен по 8 байтам так, как мы хотим, поэтому мы должны вставить ещё 7 байт набивки, чтобы снова выровнять `long`. Это также удобно обеспечивает 2-байтовое выравнивание,

которое нам нужно для последнего поля, `short`, доводя общий размер до 26 байт. Теперь, когда мы прошли через все поля, нам также нужно определить выравнивание самого `Foo`. Правило здесь — использовать наибольшее выравнивание любого из полей `Foo`, которое составит 8 байт из-за `long`. Так что, чтобы гарантировать, что `Foo` остаётся выровненным, если он размещён, скажем, в массиве, компилятор затем добавляет финальные 6 байт набивки, чтобы сделать размер `Foo` кратным его выравниванию в 32 байта.

Теперь мы готовы избавиться от наследия C и рассмотреть, что произошло бы с раскладкой, если бы мы не использовали `repr(C)` в листинге 3-1. Одно из главных ограничений представления C состоит в том, что оно требует, чтобы мы размещали все поля в том же порядке, в каком они появляются в исходном определении структуры. Представление Rust по умолчанию, `repr(Rust)`, снимает это ограничение, наряду с парой других менее значимых ограничений, таких как детерминированный порядок полей для типов, у которых случайно одинаковые поля. То есть даже два разных типа, имеющие все одни и те же поля, того же типа, в том же порядке, не гарантированно будут разложены одинаково при использовании раскладки Rust по умолчанию!

Поскольку нам теперь разрешено переупорядочивать поля, мы можем разместить их в порядке убывания размера. Это означает, что нам больше не нужна набивка между полями `Foo`; сами поля используются для достижения необходимого выравнивания! Теперь `Foo` имеет размер ровно своих полей: всего 16 байт. Это одна из причин, почему Rust по умолчанию не даёт много гарантий о том, как тип раскладывается в памяти: давая компилятору больше свободы переставлять вещи, мы можем производить более эффективный код.

Оказывается, есть также третий способ разложить тип, и это сообщить компилятору, что мы не хотим никакой набивки между нашими полями. Делая так, мы говорим, что готовы принять снижение производительности из-за использования невыровненных обращений. Наиболее распространённый сценарий использования этого — когда влияние каждого дополнительного байта памяти ощутимо, например если у вас много экземпляров типа, если у вас очень ограниченная память, или если вы отправляете представление в памяти по среде с более низкой пропускной способностью, такой как сетевое соединение. Чтобы включить это поведение, вы можете аннотировать свой тип с помощью `#[repr(packed)]`. Имейте в виду, что это может привести к гораздо более медленному коду, а в крайних случаях это может вызвать сбой вашей программы, если вы попытаетесь выполнить операции, которые процессор

поддерживает только над выровненными аргументами.

Иногда вы хотите дать конкретному полю или типу большее выравнивание, чем технически требуется. Вы можете сделать это, используя атрибут `#[repr(align(n))]`. Распространённый сценарий использования этого — гарантировать, что разные значения, хранящиеся непрерывно в памяти (например, в массиве), окажутся на разных линиях кэша (cache lines) процессора. Так вы избегаете *ложного разделения* (false sharing), которое может вызвать огромные деградации производительности в конкурентных программах. Ложное разделение происходит, когда два разных процессора обращаются к разным значениям, которые случайно разделяют одну линию кэша; хотя теоретически они могут работать параллельно, они оба в итоге соперничают за обновление одной и той же единственной записи в кэше. Мы поговорим о параллелизме гораздо подробнее в главе 11.

Сложные типы (Complex Types)

Возможно, вам любопытно, как компилятор представляет другие типы Rust в памяти. Вот краткий справочник:

- **Кортеж (Tuple)** Представлен как структура с полями того же типа, что и значения кортежа в том же порядке.
- **Массив (Array)** Представлен как непрерывная последовательность содержащегося типа без набивки между элементами.
- **Объединение (Union)** Раскладка выбирается независимо для каждого варианта. Выравнивание — максимальное среди всех вариантов.
- **Перечисление (Enumeration)** То же, что объединение, но с одним дополнительным скрытым разделяемым полем, которое хранит дискриминант варианта перечисления. Дискриминант — это значение, которое код использует для определения того, какой из вариантов перечисления содержит данное значение. Размер поля дискриминанта зависит от количества вариантов.

Динамически размеряемые типы и толстые указатели (Dynamically Sized Types and Fat Pointers)

Возможно, вам встречался маркерный трейт `Sized` в различных странных уголках документации Rust и в сообщениях об ошибках. Обычно он возникает потому, что компилятор хочет, чтобы вы предоставили тип, который является

`Sized`, а вы (по-видимому) этого не сделали. Большинство типов в Rust реализуют `Sized` автоматически — то есть у них есть размер, известный во время компиляции, — но два распространённых типа этого не делают: трейт-объекты (trait objects) и срезы (slices). Если у вас есть, например, `dyn Iterator` или `[u8]`, у них нет чётко определённого размера. Их размер зависит от некоторой информации, которая известна, когда программа выполняется, а не во время компиляции, поэтому они называются *динамически размераемыми типами* (dynamically sized types, DST). Никто не знает заранее, является ли `dyn Iterator`, который получила ваша функция, этой 200-байтовой структурой или той 8-байтовой структурой. Это создаёт проблему: часто компилятор должен знать размер чего-либо, чтобы произвести корректный код, например сколько места выделить для кортежа типа `(i32, dyn Iterator, [u8], i32)` или какое смещение использовать, если ваш код пытается обратиться к четвёртому полю. Но если тип не `Sized`, эта информация недоступна.

Компилятор требует, чтобы типы были `Sized` почти везде. Поля структур, аргументы функций, возвращаемые значения, типы переменных и типы массивов — все должны быть `Sized`. Это ограничение настолько распространено, что каждое отдельное ограничение типа, которое вы пишете, включает в себя `T: Sized`, если вы явно не откажетесь от него с помощью `T: ?Sized` (? означает «может не быть»). Но это довольно бесполезно, если у вас есть DST и вы хотите что-то с ним сделать, например если вы действительно хотите, чтобы ваша функция принимала трейт-объект или срез в качестве аргумента.

Решение этого — поместить их за *толстый указатель* (fat pointer). Толстый указатель — это просто как обычный указатель, но он включает дополнительное поле размером со слово, которое даёт дополнительную информацию об этом указателе, нужную компилятору для генерации разумного кода для работы с указателем. Когда вы берёте ссылку на DST, компилятор автоматически конструирует для вас толстый указатель. Для среза дополнительная информация — это просто длина среза. Для трейт-объекта — что ж, до этого мы доберёмся позже. И, что критически важно, этот толстый указатель *является* `Sized`. В частности, он вдвое больше размера `usize` (размера слова на целевой платформе): один `usize` для хранения указателя и один `usize` для хранения дополнительной информации, нужной, чтобы «дополнить» тип.

ПРИМЕЧАНИЕ Box и Arc также поддерживают хранение толстых указателей, поэтому они оба поддерживают `T: ?Sized`.

Трейты и ограничения трейтов (Traits and Trait Bounds)

Трейты — ключевая часть системы типов Rust; это тот клей, который позволяет типам взаимодействовать друг с другом, даже хотя они не знают друг о друге в момент своего определения. Книга «The Rust Programming Language» прекрасно справляется с освещением того, как определять и использовать трейты, поэтому я не буду повторять это здесь. Вместо этого мы рассмотрим некоторые из более технических аспектов трейтов: как они реализованы, ограничения, которых вам нужно придерживаться, и некоторые более эзотерические применения трейтов.

Компиляция и диспетчеризация (Compilation and Dispatch)

К настоящему моменту вы, вероятно, написали приличное количество обобщённого (generic) кода на Rust. Вы использовали обобщённые параметры типа на типах и методах, и, может быть, даже несколько ограничений трейтов тут и там. Но задумывались ли вы когда-нибудь, что на самом деле происходит с обобщённым кодом, когда вы компилируете, или что происходит, когда вы вызываете метод трейта на `dyn Trait`?

Когда вы пишете тип или функцию, обобщённую по `T`, вы на самом деле говорите компилятору сделать копию этого типа или функции для каждого типа `T`. Когда вы конструируете `Vec<i32>` или `HashMap<String, bool>`, компилятор по существу копирует обобщённый тип и все его блоки реализации и заменяет все вхождения каждого обобщённого параметра предоставленным конкретным типом. Он делает полную копию типа `Vec` с каждым `T`, заменённым на `i32`, и полную копию типа `HashMap` с каждым `K`, заменённым на `String`, и каждым `V` — на `bool`.

ПРИМЕЧАНИЕ В реальности компилятор не делает полную копирасту. Он копирует только те части кода, которые вы используете, так что если вы никогда не вызываете `find` на `Vec<i32>`, код для `find` не будет скопирован и скомпилирован.

То же самое применимо к обобщённым функциям. Рассмотрите код в листинге 3-2, который показывает обобщённый метод.


```
impl String {
    pub fn contains(&self, p: impl Pattern) -> bool {
        p.is_contained_in(self)
    }
}
```

Листинг 3-2: Обобщённый метод, использующий статическую диспетчеризацию

Копия этого метода делается для каждого отдельного типа паттерна (вспомните, что `impl Trait` — это сокращение для `<T: Trait>`). Нам нужна разная копия тела функции для каждого типа `impl Pattern`, потому что нам нужно знать адрес функции `is_contained_in`, чтобы её вызвать. Процессору нужно сообщить, куда перейти и продолжить выполнение. Для любого *данного* паттерна компилятор знает, что этот адрес — это адрес места, где этот тип паттерна реализует этот метод трейта. Но нет единого адреса, который мы могли бы использовать для *любого* типа, поэтому нам нужно иметь по одной копии для каждого типа, каждая со своим собственным адресом для перехода. Это называется *статической диспетчеризацией* (static dispatch), поскольку для любой данной копии метода адрес, на который мы «диспетчеризуем», известен статически.

ПРИМЕЧАНИЕ Возможно, вы заметили, что слово «статический» в этом контексте немного перегружено. «Статический» в целом используется для обозначения всего, что известно во время компиляции или может рассматриваться, как если бы оно было известно, поскольку тогда оно может быть записано в статическую память, как обсуждалось в главе 2.

Этот процесс перехода от обобщённого типа ко многим необобщённым типам называется *мономорфизацией* (monomorphization), и это часть причины, почему обобщённый код на Rust обычно работает так же хорошо, как необобщённый код. К тому времени, как компилятор начинает оптимизировать ваш код, всё выглядит так, как если бы обобщений вообще не было! Каждый экземпляр оптимизируется отдельно и со всеми известными типами. В результате код столь же эффективен, как если бы метод `is_contained_in` паттерна, который передаётся, был вызван напрямую без каких-либо присутствующих трейтов. Компилятор имеет полное знание о вовлечённых типах и может даже встроить (inline) реализацию `is_contained_in`, если захочет.

Мономорфизация также приходит со своей ценой: все эти инстанцирования вашего типа нужно скомпилировать, что может увеличить время компиляции,

если компилятор не может их оптимизировать. Каждая мономорфизированная функция также приводит к своему собственному куску машинного кода, что может сделать вашу программу больше. И поскольку инструкции не разделяются между разными инстанцированиями методов обобщённого типа, кэш инструкций процессора также становится менее эффективным, поскольку теперь ему нужно держать несколько копий по сути одних и тех же инструкций.

НЕОБОБЩЁННЫЕ ВНУТРЕННИЕ ФУНКЦИИ (NON-GENERIC INNER FUNCTIONS)

Часто значительная часть кода в обобщённом методе не зависит от типа. Рассмотрите, например, реализацию `HashMap::insert`. Код для вычисления хеша предоставленного ключа зависит от типа ключа карты, но код для обхода корзин (buckets) карты, чтобы найти точку вставки, может не зависеть. В случаях вроде этого было бы эффективнее разделять сгенерированный машинный код для необобщённых частей метода между мономорфизациями и генерировать отдельные копии только там, где это действительно нужно.

Один паттерн, который вы можете использовать для случаев вроде этого, — объявить необобщённую вспомогательную функцию внутри обобщённого метода, которая выполняет разделяемые операции. Это оставляет только зависящий от типа код, чтобы компилятор скопировал его для вас, при этом позволяя разделять вспомогательную функцию.

Сделать функцию внутренней приходит с дополнительным преимуществом, что вы не засоряете свой модуль одноцелевой функцией. Вместо этого вы можете объявить такую вспомогательную функцию вне метода; просто будьте осторожны, чтобы не сделать её методом под обобщённым блоком `impl`, так как тогда она всё равно будет мономорфизирована.

Альтернатива статической диспетчеризации — это *динамическая диспетчеризация* (dynamic dispatch), которая позволяет коду вызвать метод трейта на обобщённом типе, не зная, что это за тип. Я сказал ранее, что причина, по которой нам были нужны несколько экземпляров метода в листинге 3-2, состояла в том, что иначе ваша программа не знала бы, на какой адрес перейти, чтобы вызвать метод трейта `is_contained_in` на данном паттерне. Что ж, с динамической диспетчеризацией вызывающий просто сообщает вам это. Если вы замените `impl Pattern` на `&dyn Pattern`, вы сообщаете вызывающему, что он должен предоставить *два* куска информации для этого

аргумента: адрес паттерна *и* адрес метода `is_contained_in`. На практике вызывающий даёт вам указатель на кусок памяти, называемый таблицей виртуальных методов (virtual method table), или *vtable*, которая содержит адрес реализации *всех* методов трейта для типа, о котором идёт речь, одним из которых является `is_contained_in`. Когда код внутри метода хочет вызвать метод трейта на предоставленном паттерне, он ищет адрес реализации `is_contained_in` этого паттерна в *vtable*, а затем вызывает функцию по этому адресу. Это позволяет нам использовать одно и то же тело функции независимо от того, какой тип хочет использовать вызывающий.

ПРИМЕЧАНИЕ Каждая *vtable* также содержит информацию о раскладке и выравнивании конкретного типа, поскольку эта информация всегда нужна для работы с типом. Если вы хотите пример того, как выглядит явная *vtable*, посмотрите на тип `std::task::RawWakerVTable`.

Вы заметите, что когда мы включили динамическую диспетчеризацию с помощью ключевого слова `dyn`, нам пришлось поставить `&` перед ним. Причина в том, что мы больше не знаем во время компиляции размер типа паттерна, который передаёт вызывающий, так что мы не знаем, сколько места под него отвести. Иными словами, `dyn Trait` является `!Sized`, где `!` означает «не». Чтобы сделать его `Sized`, чтобы мы могли принять его в качестве аргумента, мы помещаем его за указатель (размер которого мы знаем). Поскольку нам также нужно передать таблицу адресов методов, этот указатель становится толстым указателем, где дополнительное слово содержит указатель на *vtable*. Вы можете использовать любой тип, способный хранить толстый указатель, для динамической диспетчеризации, например `&mut`, `Box` и `Arc`. Листинг 3-3 показывает эквивалент динамической диспетчеризации для листинга 3-2.

```
impl String {
    pub fn contains(&self, p: &dyn Pattern) -> bool {
        p.is_contained_in(&*self)
    }
}
```

Листинг 3-3: Обобщённый метод, использующий динамическую диспетчеризацию

Комбинация типа, который реализует трейт, и его *vtable* известна как *трейт-объект* (trait object). Большинство трейтов можно превратить в трейт-объекты, но не все. Например, трейт `Clone`, чей метод `clone` возвращает `Self`, не может быть превращён в трейт-объект. Если мы принимаем трейт-объект `dyn Clone`, а

затем вызываем на нём `clone`, компилятор не будет знать, какой тип вернуть. Или рассмотрите трейт `Extend` из стандартной библиотеки, у которого есть метод `extend`, обобщённый по типу предоставленного итератора (так что их может быть много экземпляров). Если бы вы хотели вызвать метод, который принимал `dyn Extend`, не было бы единого адреса для `extend`, чтобы поместить его в `vtable` трейт-объекта; пришлось бы иметь по одной записи для каждого типа, с которым `extend` мог бы когда-либо быть вызван. Это примеры трейтов, которые не являются *объектно-безопасными* (object-safe) и поэтому не могут быть превращены в трейт-объекты. Чтобы быть объектно-безопасным, ни один из методов трейта не может быть обобщённым или использовать тип `Self`. Кроме того, трейт не может иметь никаких статических методов (то есть методов, чей первый аргумент не разыменовывается до `Self`), поскольку было бы невозможно узнать, какой экземпляр метода вызывать. Неясно, например, какой код должен выполнить `FromIterator::from_iter(&[0])`.

Читая о трейт-объектах, вы можете встретить упоминания ограничения трейта `Self: Sized`. Такое ограничение подразумевает, что `Self` не используется через трейт-объект (поскольку тогда он был бы `!Sized`). Вы можете поместить это ограничение на трейт, чтобы потребовать, чтобы трейт никогда не использовал динамическую диспетчеризацию, или вы можете поместить его на конкретный метод, чтобы сделать этот метод недоступным, когда к трейту обращаются через трейт-объект. Методы с ограничением `where Self: Sized` исключаются при проверке, является ли трейт объектно-безопасным.

Динамическая диспетчеризация сокращает время компиляции, поскольку больше не нужно компилировать несколько копий типов и методов, и она может улучшить эффективность кэша инструкций вашего процессора. Однако она также мешает компилятору оптимизировать для конкретных используемых типов. При динамической диспетчеризации всё, что компилятор может сделать для `find` в листинге 3-2, — это вставить вызов функции через `vtable`; он больше не может выполнять никаких дополнительных оптимизаций, поскольку не знает, какой код будет находиться по другую сторону этого вызова функции. Кроме того, каждый вызов метода на трейт-объекте требует поиска в `vtable`, что добавляет небольшое количество накладных расходов по сравнению с прямым вызовом метода.

Когда вам предоставлен выбор между статической и динамической диспетчеризацией, редко есть однозначно правильный ответ. В широком смысле, однако, вы захотите использовать статическую диспетчеризацию в

своих библиотеках и динамическую диспетчеризацию в своих исполняемых файлах (binaries). В библиотеке вы хотите позволить вашим пользователям решать, какой вид диспетчеризации лучше для них, поскольку вы не знаете их потребностей. Если вы используете динамическую диспетчеризацию, они вынуждены делать то же, тогда как если вы используете статическую диспетчеризацию, они могут выбирать, использовать динамическую диспетчеризацию или нет. В исполняемом файле, с другой стороны, вы пишете финальный код, так что нет нужд, кроме тех, что есть у кода, который вы пишете. Динамическая диспетчеризация часто позволяет писать более чистый код, опускающий обобщённые параметры, и компилируется быстрее, всё это за (обычно) маргинальную цену в производительности, так что это часто лучший выбор для исполняемых файлов.

Обобщённые трейты (Generic Traits)

Трейты Rust могут быть обобщёнными одним из двух способов: с обобщёнными параметрами типа, как `trait Foo<T>`, или со связанными типами (associated types), как `trait Foo { type Bar; }`. Разница между ними не сразу очевидна, но, к счастью, эмпирическое правило довольно простое: используйте связанный тип, если вы ожидаете только одну реализацию трейта для данного типа, и используйте обобщённый параметр типа в противном случае.

Обоснование этого в том, что со связанными типами часто значительно легче работать, но они не позволят множественные реализации. Так что, проще говоря, совет действительно в том, чтобы просто использовать связанные типы всякий раз, когда можете.

С обобщённым трейтом пользователи всегда должны указывать все обобщённые параметры и повторять любые ограничения на этих параметрах. Это может быстро стать запутанным и трудным в сопровождении. Если вы добавляете обобщённый параметр к трейту, все пользователи этого трейта также должны быть обновлены, чтобы отразить это изменение. И поскольку для данного типа может существовать несколько реализаций трейта, компилятору может быть трудно решить, какой экземпляр трейта вы имели в виду, что приводит к ужасным разрешающим неоднозначность вызовам функций вроде `FromIterator::<u32>::from_iter`. Но плюс в том, что вы можете реализовать трейт несколько раз для одного типа — например, вы можете реализовать `PartialEq` для нескольких правых сторон для вашего типа, или вы можете реализовать и `FromIterator<T>`, и `FromIterator<&T> where T: Clone`, именно

благодаря гибкости, которую предоставляют обобщённые трейты.

Со связанными типами, с другой стороны, компилятору нужно знать только тип, который реализует трейт, и все связанные типы следуют из этого (поскольку есть только одна реализация). Это означает, что ограничения могут жить в самом трейте и не должны повторяться при использовании. В свою очередь, это позволяет трейту добавлять дополнительные связанные типы, не затрагивая его пользователей. И поскольку тип диктует все связанные типы трейта, вам никогда не нужно разрешать неоднозначность с помощью синтаксиса вызова с полностью определённым именем (unified function calling syntax), показанного в предыдущем абзаце. Однако вы не можете реализовать `Deref` для нескольких типов `Target`, ни реализовать `Iterator` с несколькими разными типами `Item`.

Когерентность и правило сироты (Coherence and the Orphan Rule)

В Rust есть довольно строгие правила о том, где вы можете реализовывать трейты и на каких типах вы можете их реализовывать. Эти правила существуют, чтобы сохранить свойство *когерентности* (coherence): для любого данного типа и метода есть всегда только один корректный выбор того, какую реализацию метода использовать для этого типа. Чтобы понять, почему это важно, рассмотрите, что произошло бы, если бы я мог написать свою собственную реализацию трейта `Display` для типа `bool` из стандартной библиотеки. Теперь, для любого кода, который пытается напечатать значение `bool` и включает мой крейт, компилятор не знал бы, выбрать ли реализацию, которую написал я, или ту, что из стандартной библиотеки. Ни один из выборов не корректен или лучше другого, и компилятор, очевидно, не может выбирать случайно. Та же проблема возникает, если стандартная библиотека вообще не вовлечена, но вместо этого у нас есть два крейта, которые зависят друг от друга, и они оба реализуют трейт для некоторого разделяемого типа. Свойство когерентности гарантирует, что компилятор никогда не оказывается в этих ситуациях и никогда не должен делать эти выборы: всегда будет ровно один очевидный выбор.

Простой способ поддержать когерентность был бы в том, чтобы гарантировать, что только крейт, который определяет трейт, может писать реализации для этого трейта; если никто другой не может реализовать трейт, то нигде не может быть конфликтующих реализаций. Однако это слишком ограничительно на практике и по существу сделало бы трейты бесполезными, поскольку не

было бы способа реализовать трейты вроде `std::fmt::Debug` и `serde::Serialize` для ваших собственных типов, если только вы не включили бы свой собственный тип в определяющий крейт. Противоположная крайность, говорящая, что вы можете реализовывать трейты только для своих собственных типов, решает эту проблему, но вводит другую: крейт, который определяет трейт, теперь не может предоставить реализации этого трейта для типов в стандартной библиотеке или других популярных крейтах! В идеале мы хотели бы найти некий набор правил, который балансирует желание нижестоящих (downstream) крейтов реализовывать вышестоящие (upstream) трейты для своих собственных типов против желания вышестоящих крейтов иметь возможность добавлять реализации своих собственных трейтов, не ломая нижестоящий код.

ПРИМЕЧАНИЕ Вышестоящее (upstream) относится к тому, от чего зависит ваш код, а нижестоящее (downstream) — к тому, что зависит от вашего кода. Часто эти термины используются в прямом смысле зависимостей крейтов, но они также могут использоваться для обозначения авторитетных форков (forks) кодовой базы — если вы делаете форк компилятора Rust, официальный компилятор Rust — ваш «upstream».

В Rust правило, которое устанавливает этот баланс, — это *правило сироты* (orphan rule). Проще говоря, правило сироты гласит, что вы можете реализовать трейт для типа только если либо трейт, либо тип локальны для вашего крейта. Так, вы можете реализовать `Debug` для своего собственного типа, и вы можете реализовать `MyNeatTrait` для `bool`, но вы не можете реализовать `Debug` для `bool`. Если попробуете, ваш код не скомпилируется, и компилятор скажет вам, что есть конфликтующие реализации.

Это уводит вас довольно далеко; это позволяет вам реализовывать свои собственные трейты для сторонних типов и реализовывать сторонние трейты для своих собственных типов. Однако правило сироты — не конец истории. Есть ряд дополнительных следствий, оговорок и исключений из него, о которых вам следует знать.

Бланкетные реализации (Blanket Implementations)

Правило сироты позволяет вам реализовывать трейты для диапазона типов с кодом вроде `impl<T> MyTrait for T where T: ...` и так далее. Это *бланкетная реализация* (blanket implementation) — она не ограничена одним конкретным

типом, а вместо этого применяется к широкому диапазону типов. Только крейт, который определяет трейт, имеет право писать бланкетную реализацию, и добавление бланкетной реализации к существующему трейту считается ломающим изменением (breaking change). Если бы это было не так, нижестоящий крейт, содержащий `impl MyTrait for Foo`, мог бы внезапно перестать компилироваться только потому, что вы обновили крейт, который определяет `MyTrait`, с ошибкой о конфликтующей реализации.

Фундаментальные типы (Fundamental Types)

Некоторые типы настолько существенны, что необходимо позволять кому угодно реализовывать трейты для них, даже если это, казалось бы, нарушает правило сироты. Эти типы помечены атрибутом `#[fundamental]` и в настоящее время включают `&`, `&mut` и `Box`. Для целей правила сироты фундаментальные типы как будто не существуют — они фактически стираются перед проверкой правила сироты, чтобы позволить вам, например, реализовать `IntoIterator` для `&MyType`. С одним лишь правилом сироты эта реализация не была бы разрешена, поскольку она реализует сторонний трейт для стороннего типа — `IntoIterator` и `&` оба приходят из стандартной библиотеки. Добавление бланкетной реализации для фундаментального типа также считается ломающим изменением.

Покрытые реализации (Covered Implementations)

Есть некоторые ограниченные случаи, когда мы хотим позволить реализацию стороннего трейта для стороннего типа, что правило сироты обычно не допускает. Простейший пример этого — когда вы хотите написать что-то вроде `impl From<MyType> for Vec<i32>`. Здесь трейт `From` сторонний, как и тип `Vec`, и всё же нет опасности нарушить когерентность. Это потому, что конфликтующая реализация могла бы быть добавлена только через бланкетную реализацию в стандартной библиотеке (стандартная библиотека не может иначе именовать `MyType`), что в любом случае является ломающим изменением.

Чтобы позволить такие виды реализаций, правило сироты включает узкое исключение, которое разрешает реализовывать сторонние трейты для сторонних типов при очень специфическом наборе обстоятельств. В частности, данный `impl<P1..=Pn> ForeignTrait<T1..=Tn> for T0` разрешён только если хотя бы один `Ti` является локальным типом и никакой `T` перед первым таким `Ti` не является одним из обобщённых типов `P1..=Pn`. Обобщённым параметрам типа (`P`) *разрешено* появляться в `T0..Ti`, если они *покрыты* (covered) каким-то

промежуточным типом. Тип `T` покрыт, если он появляется как параметр типа некоторого другого типа (например, `Vec<T>`), но не если он стоит сам по себе (просто `T`) или просто появляется за фундаментальным типом вроде `&T`. Так, все реализации в листинге 3-4 допустимы.

```
impl<T> From<T> for MyType
impl<T> From<T> for MyType<T>
impl<T> From<MyType> for Vec<T>
impl<T> ForeignTrait<MyType, T> for Vec<T>
```

Листинг 3-4: Допустимые реализации сторонних трейтов для сторонних типов

Однако реализации в листинге 3-5 недопустимы.

```
impl<T> ForeignTrait for T
impl<T> From<T> for T
impl<T> From<Vec<T>> for T
impl<T> From<MyType<T>> for T
impl<T> From<T> for Vec<T>
impl<T> ForeignTrait<T, MyType> for Vec<T>
```

Листинг 3-5: Недопустимые реализации сторонних трейтов для сторонних типов

Это послабление правила сироты усложняет правила того, что составляет ломающее изменение, когда вы добавляете новую реализацию для существующего трейта. В частности, добавление реализации к существующему трейту неломаящее только если оно содержит хотя бы один *новый* локальный тип, и этот новый локальный тип удовлетворяет правилам исключения, описанным ранее. Добавление любой другой новой реализации является ломающим изменением.

ПРИМЕЧАНИЕ Заметьте, что `impl<T> ForeignTrait<LocalType, T> for ForeignType` допустим, но `impl<T> ForeignTrait<T, LocalType> for ForeignType` — нет! Это может показаться произвольным, но без этого правила вы могли бы написать `impl<T> ForeignTrait<T, LocalType> for ForeignType`, а другой крейт мог бы написать `impl<T> ForeignTrait<TheirType, T> for ForeignType`, и конфликт возник бы только когда два крейта были бы сведены вместе. Вместо того чтобы запрещать этот паттерн вообще, правило сироты требует, чтобы ваш локальный тип шёл перед параметром типа, что разрывает связь и гарантирует, что если оба крейта поддерживают когерентность в изоляции, они также поддерживают её при объединении.

Ограничения трейтов (Trait Bounds)

Стандартная библиотека изобилует ограничениями трейтов, будь то то, что ключи в `HashMap` должны реализовывать `Hash + Eq`, или то, что функция, переданная в `thread::spawn`, должна быть `FnOnce + Send + 'static`. Когда вы пишете обобщённый код сами, он почти наверняка будет включать ограничения трейтов, поскольку иначе ваш код мало что может сделать с типом, по которому он обобщён. По мере того как вы пишете более сложные обобщённые реализации, вы обнаружите, что вам также нужна большая точность от ваших ограничений трейтов, так что давайте рассмотрим некоторые из способов этого достичь.

Прежде всего, ограничения трейтов не обязаны быть в форме `T: Trait`, где `T` — некоторый тип, по которому ваша реализация или тип обобщены. Ограничения могут быть произвольными ограничениями типов и даже не должны включать обобщённые параметры, типы аргументов или локальные типы. Вы можете написать ограничение трейта вроде `where String: Clone`, даже хотя `String: Clone` всегда истинно и не содержит локальных типов. Вы также можете написать `where io::Error: From<MyError<T>>`; ваши обобщённые параметры типа не обязаны появляться только в левой части. Это не только позволяет вам выражать более сложные ограничения, но и может избавить вас от ненужного повторения ограничений. Например, если ваш метод хочет сконструировать `HashMap<K, V, S>`, чьи ключи — некоторый обобщённый тип `T`, а значение — `usize`, вместо того чтобы выписывать ограничения вроде `where T: Hash + Eq, S: BuildHasher + Default`, вы могли бы написать `where HashMap<T, usize, S>: FromIterator`. Это избавляет вас от необходимости искать точные требования ограничений для методов, которые вы в итоге используете, и более ясно сообщает «истинное» требование вашего кода. Как вы видите, это также может значительно снизить сложность ваших ограничений, если ограничения на нижележащих методах трейтов, которые вы хотите вызвать, сложны.

ТРЕЙТ DERIVE (DERIVE TRAIT)

Хотя `#[derive(Trait)]` чрезвычайно удобен, в контексте ограничений трейтов вам следует знать об одной тонкости касательно того, как он часто реализован. Многие раскрытия `#[derive(Trait)]` десакhariзуются в `impl Trait for Foo<T> where T: Trait`. Это часто то, что вы хотите, но не всегда. Например, рассмотрите, что произойдет, если мы попытаемся вывести (derive) `Clone` таким способом для `Foo<T>`, а `Foo` содержит `Arc<T>`. `Arc` реализует `Clone` *независимо* от того, реализует ли `T` `Clone`, но из-за выведенного ограничения `Foo` будет реализовывать `Clone`, только если это делает `T`! Обычно это не слишком большая проблема, но это добавляет ограничение там, где оно не нужно. Если мы переименуем тип в `Shared`, проблема может стать немного яснее. Представьте, как запутается пользователь, у которого есть `Shared<NotClone>`, когда компилятор скажет ему, что он не может это клонировать! На момент написания именно так работает `#[derive(Clone)]`, предоставляемый стандартной библиотекой, хотя это может измениться в будущем.

Иногда вам нужны ограничения на связанные типы тех типов, по которым вы обобщены. В качестве примера рассмотрите метод итератора `flatten`, который принимает итератор, производящий элементы, которые сами в свою очередь реализуют `Iterator`, и производит итератор элементов этих внутренних итераторов. Тип, который он производит, `Flatten`, обобщен по `I`, который является типом внешнего итератора. `Flatten` реализует `Iterator`, если `I` реализует `Iterator` и элементы, выдаваемые `I`, сами реализуют `IntoIterator`. Чтобы позволить вам писать ограничения вроде этого, Rust позволяет ссылаться на связанные типы типа, используя синтаксис `Type::AssocType`. Например, мы можем сослаться на тип `Item I`, используя `I::Item`. Если у типа несколько связанных типов с одинаковым именем, например если трейт, который предоставляет связанный тип, сам обобщен (и поэтому есть много реализаций), вы можете разрешить неоднозначность с помощью синтаксиса `<Type as Trait>::AssocType`. Используя это, вы можете писать ограничения не только для типа внешнего итератора, но и для типа элемента этого внешнего итератора.

В коде, который интенсивно использует обобщения, вы можете обнаружить, что вам нужно написать ограничение, которое говорит о ссылках на тип. Обычно это нормально, поскольку у вас, как правило, будет обобщенный параметр времени жизни, который вы можете использовать как время жизни для этих ссылок. В некоторых случаях, однако, вы хотите, чтобы ограничение говорило «эта ссылка реализует этот трейт для любого времени жизни». Это крайне редко — на момент написания стандартная библиотека использует это

всего в трёх местах, — но это случается. Этот тип ограничения известен как *ограничение трейта высшего ранга* (higher-ranked trait bound), и он особенно полезен в связке с трейтами `Fn`. Например, скажем, вы хотите быть обобщённым по функции, которая принимает ссылку на `T` и возвращает ссылку на *внутри* этого `T`. Если вы напишете `F: Fn(&T) -> &U`, вам нужно предоставить время жизни для этих ссылок, но вы действительно хотите сказать «любое время жизни, пока выход тот же, что и вход». Используя время жизни высшего ранга, вы можете написать `F: for<'a> Fn(&'a T) -> &'a U`, чтобы сказать, что для *любого* времени жизни `'a` ограничение должно выполняться. Компилятор Rust достаточно умен, чтобы автоматически добавлять `for`, когда вы пишете ограничения `Fn` со ссылками вроде этого, что является частью причины, почему вам так редко нужна явная форма, но об этом стоит знать.

Чтобы свести всё это воедино, рассмотрите код в листинге 3-6, который можно использовать для реализации `Debug` для любого типа, который можно итерировать и чьи элементы являются `Debug`.

```
impl Debug for AnyIterable
where for<'a> &'a Self: IntoIterator,
      for<'a> <&'a Self as IntoIterator>::Item: Debug {
    fn fmt(&self, f: &mut Formatter) -> Result<(), Error> {
        f.debug_list().entries(self).finish()
    }
}
```

Листинг 3-6: Чрезмерно обобщённая реализация `Debug` для любой итерируемой коллекции

Вы могли бы скопировать эту реализацию практически для любого типа коллекции, и она «просто работала бы». Конечно, вы можете захотеть более умную реализацию `debug`, но это весьма хорошо иллюстрирует мощь ограничений трейтов.

Маркерные трейты (Marker Traits)

Обычно мы используем трейты для обозначения функциональности, которую могут поддерживать несколько типов; тип `Hash` можно хешировать, вызвав `hash`, тип `Clone` можно клонировать, вызвав `clone`, а тип `Debug` можно отформатировать для отладки, вызвав `fmt`. Но не все трейты функциональны в этом смысле. Некоторые трейты, называемые *маркерными трейтами* (marker traits), вместо этого указывают свойство реализующего типа. У маркерных трейтов нет

методов или связанных типов, и они служат лишь для того, чтобы сообщить вам, что определённый тип можно использовать определённым образом. Например, если тип реализует маркерный трейт `Send`, его безопасно отправлять через границы потоков. Если он не реализует этот маркерный трейт, его небезопасно отправлять. С этим поведением не связано никаких методов; это просто факт о типе. В стандартной библиотеке есть ряд таких в модуле `std::marker`, включая `Send`, `Sync`, `Copy`, `Sized` и `Unpin`. Большинство из них (все, кроме `Copy`) также являются *авто-трейтами* (auto-traits); компилятор автоматически реализует их для типов, если только тип не содержит чего-то, что не реализует этот маркерный трейт.

Маркерные трейты служат важной цели в Rust: они позволяют вам писать ограничения, которые захватывают семантические требования, прямо не выраженные в коде. Нет вызова `send` для типа, который требует, чтобы тип был `Send`. Вместо этого код *предполагает*, что данный тип нормально использовать в отдельном потоке, и без маркерных трейтов компилятору не было бы способа проверить это предположение. Программисту пришлось бы помнить это предположение и очень внимательно читать код, что, как мы все знаем, не то, на что нам хотелось бы полагаться. Этот путь усеян гонками данных (data races), сегфолтами (segfaults) и другими проблемами времени выполнения.

Похожи на маркерные трейты *маркерные типы* (marker types). Это типы-единицы (unit types) вроде `struct MyMarker;`, которые не содержат данных и не имеют методов. Маркерные типы полезны для, ну, маркировки типа как находящегося в определённом состоянии. Они пригождаются, когда вы хотите сделать невозможным для пользователя неправильно использовать API. Например, рассмотрите тип вроде `SshConnection`, который может быть аутентифицирован, а может и нет. Вы могли бы добавить обобщённый аргумент типа к `SshConnection`, а затем создать два маркерных типа: `Unauthenticated` и `Authenticated`. Когда пользователь впервые подключается, он получает `SshConnection<Unauthenticated>`. В этом блоке `impl` вы предоставляете единственный метод: `connect`. Метод `connect` возвращает `SshConnection<Authenticated>`, и только в этом блоке `impl` вы предоставляете остальные методы для выполнения команд и тому подобного. Мы рассмотрим этот паттерн подробнее в главе 4.

Экзистенциальные типы (Existential Types)

В Rust вам очень редко нужно указывать типы переменных, которые вы

объявляете в теле функции, или типы обобщённых аргументов к методам, которые вы вызываете. Это благодаря *выводу типов* (type inference), когда компилятор решает, какой тип использовать, на основе того, во что вычисляется тип в коде, в котором этот тип появляется. Компилятор обычно выводит типы только для переменных и для аргументов (и возвращаемых типов) замыканий (closures); определения верхнего уровня вроде функций, типов, трейтов и блоков реализации трейтов — все требуют, чтобы вы явно именовали все типы. Для этого есть пара причин, но основная в том, что вывод типов гораздо легче, когда у вас есть хотя бы некоторые известные точки, с которых начать вывод. Однако не всегда легко или даже возможно полностью именовать тип! Например, если вы возвращаете замыкание из функции или `async`-блок из метода трейта, его тип не имеет имени, которое вы можете вписать в свой код.

Чтобы справиться с такими ситуациями, Rust поддерживает *экзистенциальные типы* (existential types). Скорее всего, вы уже видели экзистенциальные типы в действии. Все функции, помеченные как `async fn` или с возвращаемым типом `impl Trait`, имеют экзистенциальный возвращаемый тип: сигнатура не даёт истинный тип возвращаемого значения, лишь намёк, что функция возвращает *некоторый* тип, реализующий некоторый набор трейтов, на которые вызывающий может полагаться. И, что критически важно, вызывающий может полагаться только на возвращаемый тип, реализующий эти трейты, и ничего больше.

ПРИМЕЧАНИЕ Технически не вполне верно, что вызывающий полагается на возвращаемый тип и ничего больше. Компилятор также будет распространять авто-трейты вроде `Send` и `Sync` через `impl Trait` в позиции возврата. Мы рассмотрим это подробнее в следующей главе.

Это поведение и есть то, что даёт экзистенциальным типам их название: мы утверждаем, что существует некоторый конкретный тип, соответствующий сигнатуре, и оставляем компилятору выяснить, что это за тип. Компилятор обычно затем выясняет это, применяя вывод типов к телу функции.

Не все экземпляры `impl Trait` используют экзистенциальные типы. Если вы используете `impl Trait` в позиции аргумента функции, это на самом деле просто сокращение для неименованного обобщённого параметра этой функции. Например, `fn foo(s: impl ToString)` — это в основном просто синтаксический сахар для `fn foo<S: ToString>(s: S)`.

Экзистенциальные типы пригождаются особенно когда вы реализуете трейты, у которых есть связанные типы. Например, представьте, что вы реализуете трейт `IntoIterator`. У него есть связанный тип `IntoIter`, который содержит тип итератора, в который можно превратить данный тип. С экзистенциальными типами вам не нужно определять отдельный тип итератора, чтобы использовать его для `IntoIter`. Вместо этого вы можете задать связанный тип как `impl Iterator<Item = Self::Item>` и просто написать выражение внутри `fn into_iter(self)`, которое вычисляется в `Iterator`, например используя `map` и `filter` над некоторым существующим типом итератора.

Экзистенциальные типы также предоставляют возможность за пределами простого удобства: они позволяют вам выполнять стирание типов с нулевой стоимостью (*zero-cost type erasure*). Вместо экспорта вспомогательных типов только потому, что они появляются где-то в публичной сигнатуре — итераторы и `futures` являются распространёнными примерами этого, — вы можете использовать экзистенциальные типы, чтобы скрыть нижележащий конкретный тип. Пользователям вашего интерфейса показаны только трейты, которые реализует соответствующий тип, тогда как конкретный тип оставлен как деталь реализации. Это не только упрощает интерфейс, но и позволяет вам менять эту реализацию как пожелаете, не ломая нижестоящий код в будущем.

Итоги (Summary)

Эта глава предоставила тщательный обзор системы типов Rust. Мы рассмотрели как то, как компилятор материализует типы в памяти, так и то, как он рассуждает о самих типах. Это важный фоновый материал для написания небезопасного кода, сложных интерфейсов приложений и асинхронного кода в последующих главах. Вы также обнаружите, что многие из рассуждений о типах из этой главы играют роль в том, как вы проектируете интерфейсы кода на Rust, что мы рассмотрим в следующей главе.